

Práctica 4: CWS2

1. Objetivos

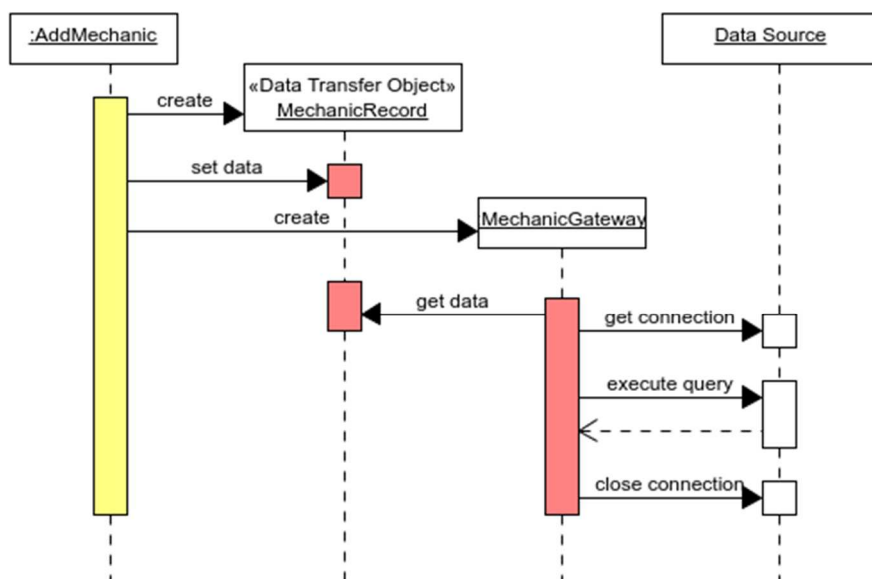
- Separar el código de la capa de negocio en dos capas: lógica de negocio y persistencia.
- Organizar la capa de persistencia utilizando los patrones TDG, Facade y Simple Factory.
- Organizar la capa service utilizando los patrones Transaction Script y Command.

2. Patrón capas para separar lógica de negocio y acceso a datos

Crea un paquete **cws.application.persistence** para contener el código de acceso a datos, y **refactoriza** las clases de la capa **service** moviendo **persistence** dicho código.

3. Implementación del patrón Table Data Gateway en la capa de acceso a datos

El patrón **Table Data Gateway (TDG)** permite que los objetos de negocio trabajen con la base de datos sin acceder a ella directamente. En lugar de eso, utilizan objetos intermedios llamados table data gateways. Cada **TDG** contiene el código necesario para realizar operaciones sobre una tabla concreta, como buscar, insertar, actualizar o eliminar registros. Así, los objetos de negocio solo tienen que llamar a los métodos del TDG para guardar o recuperar la información que necesitan.



Estructura de paquetes

Paquete **persistence**: incluye todo el código de acceso a datos. Contiene la interfaz **Gateway.java** (ver código adicional) que define operaciones comunes sobre cualquier tabla (insertar, borrar, recuperar o actualizar).

Subpaquetes **para cada tabla**: Contienen el código de acceso a esa tabla concreta.

Norma de estilo: Como norma general, los nombres de los subpaquetes será el mismo que las tablas en la base de datos, en singular y sin la T inicial.

Cada uno contendrá una interfaz que extiende Gateway **añadiendo métodos únicamente de recuperación de datos**. Por ejemplo, **MechanicGateway** puede añadir el método **findByNif**.



Además, un subpaquete **impl** que contendrá las clases que implementan el gateway.

Norma de estilo: El nombre de esas clases será el mismo que las interfaces con el sufijo **Impl**.

Implementación de los objetos TDG

Recuperación de información: los métodos de lectura recuperarán filas de la tabla en forma de **ResultSet**, y los transformarán en objetos **dto** antes de enviarlos a la capa **service**. Estos **DTO** se declaran dentro de cada **TDG**. Para evitar confundir los **DTO** de **service** y **persistence**, añadiremos el sufijo **Record** a los **DTO** de la capa **persistence**. Éstos son una copia de las filas de la base de datos (incluyendo **entityState**, **createdAt** y **updatedAt** lo cual nos obligará a reescribir sentencias **sql**).

Norma de estilo: Como norma general, los nombres de estos **DTOs** serán <nombre de la tabla en singular y sin la T inicial><Record> (**MechanicRecord**, **InvoiceRecord**, ...). Los nombres de los campos serán exactamente los nombres de los campos de las tablas, todo en minúscula.

Modificación de información: para añadir y actualizar filas de la tabla, la capa **service** debe enviar información a la capa **persistence** transformando sus propios objetos **dto** en objetos **record**.

Clases Assembler: las transformaciones **ResultSet** a **record** y **dto** a **record** o viceversa, hacen el código más extenso y difícil de leer. Para evitarlo, se crearán clases llamadas **Assembler** (por ejemplo, **MechanicAssembler**), que contendrán los métodos encargados de realizar dichas conversiones. Estas clases se ubicarán en los paquetes **impl** (dentro de la capa de **persistence**) y **crud** (dentro de la capa de **servicio**).

Excepciones: los métodos de los objetos **TDG** pueden lanzar la excepción **SQLException**, estrechamente vinculada a detalles específicos del motor de base de datos. Para **desacoplar la lógica de negocio de la tecnología de persistencia**, se define una nueva excepción llamada **PersistenceException** (disponible en código extra). De este modo, los métodos de los **TDG** no propagarán directamente **SQLException**, sino que **lanzarán PersistenceException**, manteniendo así la capa de negocio independiente de las particularidades de la base de datos.

3.1. Extraer del código las líneas SQL de acceso a datos

El fichero **queries.properties** contendrá todas las sentencias **SQL** que se encuentren en el código. Comprueba que las que ya se encuentran allí son correctas.

Este fichero debe estar organizado como sigue:

- Todas las sentencias que se refieran a la misma tabla irán juntas, precedidas por una línea de comentario que indique el nombre de la tabla.
- Las sentencias se ordenarán alfabéticamente en el fichero (**TCLIENTS_***, irá antes que **TVEHICLES_***).
- El nombre de las propiedades debe ser todo en mayúsculas con el formato:

Nombre de la tabla_Nombre del método del Gateway que usa la sentencia

Utiliza la clase **Queries** proporcionada para leer las sentencias de este fichero.

3.2. Patrón Simple Factory

Se proporciona la clase **PersistenceFactory** para **usarse desde la capa de negocio** y así evitar referencias a objetos de implementación de la capa de acceso a datos. Para acceder a ella se utilizará la clase **Factories**.

Comenta todas las líneas de código (incluyendo imports) excepto las que se refieran a mecánicos, facturas, intervenciones y workorders. **Descoméntalas** a medida que las necesites.

4. Implementación del patrón Transaction Script en la capa de servicio

La lógica de negocio se organiza como procedimientos, uno por cada acción solicitada por algún cliente (borrar mecánico, generar factura, generar nóminas, etc). Estos procedimientos se llaman **Transaction Script**. AddMechanic, DeleteMechanic, ... serán TS.

Cada Transaction Script (**TS**) debe ejecutarse de forma **transaccional**. Por tanto, es necesario que **todos** los métodos de los gateways que se invoquen en un TS utilicen **la misma conexión**, garantizando así una única transacción. Esto obliga que que **la gestión de la conexión y la transacción** se realice en el propio script, lo que da sentido al nombre **Transaction Script**.

4.1. Reescribir métodos de TDG y TS

La clase **uo.ri.util.jdbc** tiene los métodos:

- **createThreadConnection**: crea una conexión y la guarda en el hilo de ejecución.
- **getCurrentConnection**: recupera la conexión guardada en el método anterior.

Los **transaction scripts** quedarían como sigue:

- Obtener conexión con la base de datos utilizando **createThreadConnection**.
- Desactivar el modo **autocommit**.
- Realizar las operaciones pertinentes utilizando, si fuese necesario, los gateways.
- **Confirmar** la transacción.
- Retornar resultado (si lo hubiera) a la capa de presentación.
- En caso de que se produzca alguna excepción durante la ejecución, es necesario realizar **rollback**. Si la excepción es **SQLException**, transformar en **RuntimeException**.

Y los **gateway**:

- **Recuperan** la conexión, en lugar de crearla.
- No tienen más cambios.

5. Implementación del patrón Command en la capa de servicio

Para mejorar la capa de negocio se utiliza el patrón **Command**. Algunos de los roles de este patrón se proporcionan ya implementados (todos pueden encontrarse en el material suplementario):

- **Command interface**: se proporciona la interfaz **Command** que los TS deben implementar.
- **Invoker**: la clase **CommandExecutor** ejecuta los comandos realizando las acciones comunes.

Además:

- La clase **Jdbc** debe **moverse** a **uo.ri.cws.application.persistence.util.executor**.
- Las implementaciones de las interfaces de servicio crean y configuran objetos de tipo **command**. Los ejecuta el comando utilizando el **CommandExecutor**.
- **Concrete Command** y **Receiver**: realizan sólo acciones de negocio.

6. Repetir los patrones anteriores con el caso generar factura

Como **trabajo autónomo**, aplica los mismos patrones a **facturar trabajos** y **buscar trabajos no facturados por cliente**.